# Laporan Pemrograman Mobile 5




**Nama** : **David Yehezkiel**
**NIM** : **20240801013**
**Prodi** : **Teknik Informatika**

**Universitas Esa Unggul**
**Kampus Citra Raya**
**2026**

Pertanyaan Refleksi

1. Mengapa shared_preferences kurang cocok untuk menyimpan ratusan catatan? Apa pilihan yang lebih baik?
2. Mengapa query memakai placeholder ? dan whereArgs, bukan menyambung teks langsung?
3. Mengapa setelah kembali dari halaman form, daftar perlu dimuat ulang? Bagaimana caranya pada kode ini?
4. Apa fungsi onCreate dan onUpgrade, dan kapan masing-masing dijalankan?

Jawab :

1. shared_preferences dirancang untuk data kecil dengan key value sehingga menyimpan ratusan catatan akan menyulitkan proses pencarian. Lebih baik menggunakan basis data seperti SQLite yang memang dirancang untuk data yang banyak.
2. Placeholder digunakan sebagai praktik keamanan untuk mencegah SQL Injection.
3. Karena data dalam halaman form telah berubah dan tanpa memuat ulang data, ui di halaman tidak akan muncul. Cara pakainya dengan “await navigator.push()” saat membuka halaman form, kode akan di pause hingga halaman ditutup, setelah push selesai panggil kembali fungsi dengan “_muat()” yang mengupdate _future.
4. onCreate berfungsi untuk menjalankan perintah inisiasi struktur basis data dan dijalankan saat basis data belum ada di perangkat. onUpgrade berfungsi untuk mengupdate skema basis data dan dipakai saat nomor versi basis data yang baru lebih tinggi daripada yang tersimpan di perangkat.

## Bagian A :

```
PS C:\Users\advan\StudioProjects\untitled> flutter pub add shared_preferences
```

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
 const MyApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     title: 'Praktikum 5',
     home: const PengaturanPage(),
   );
 }
}

class PengaturanPage extends StatefulWidget {
 const PengaturanPage({super.key});

 @override
 State<PengaturanPage> createState() => _PengaturanPageState();
}

class _PengaturanPageState extends State<PengaturanPage> {
 final _controller = TextEditingController();
 bool _gelap = false;
 int _bukaKe = 0;

 @override
 void initState() {
   super.initState();
   _muat();
 }

 @override
 void dispose() {
   _controller.dispose();
   super.dispose();
 }

 Future<void> _muat() async {
   final prefs = await SharedPreferences.getInstance();
   if (!mounted) return;
   setState(() {
     _controller.text = prefs.getString('nama') ?? '';
     _gelap = prefs.getBool('gelap') ?? false;
     _bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;
   });
   await prefs.setInt('bukaKe', _bukaKe);
```

```dart
}

Future<void> _simpanNama() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('nama', _controller.text.trim());
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Nama disimpan')),
  );
}

Future<void> _ubahTema(bool nilai) async {
  setState(() => _gelap = nilai);
  final prefs = await SharedPreferences.getInstance();
  await prefs.setBool('gelap', nilai);
}

@override
Widget build(BuildContext context) {
  return Theme(
    data: ThemeData(
      brightness: _gelap ? Brightness.dark : Brightness.light,
      colorSchemeSeed: Colors.blue,
      useMaterial3: true,
    ),
    child: Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: _simpanNama,
              child: const Text('Simpan nama'),
            ),
            SwitchListTile(
              title: const Text('Mode gelap'),
              value: _gelap,
              onChanged: _ubahTema,
            ),
            const SizedBox(height: 12),
            Text('Aplikasi dibuka ke-$_bukaKe kali'),
          ],
        ),
      ),
```

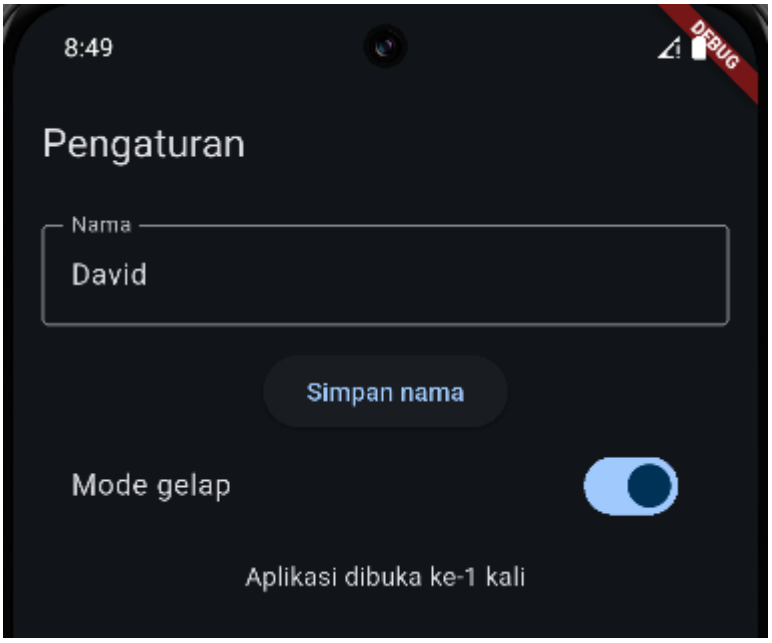
```
      ),
    );
  }
}
```

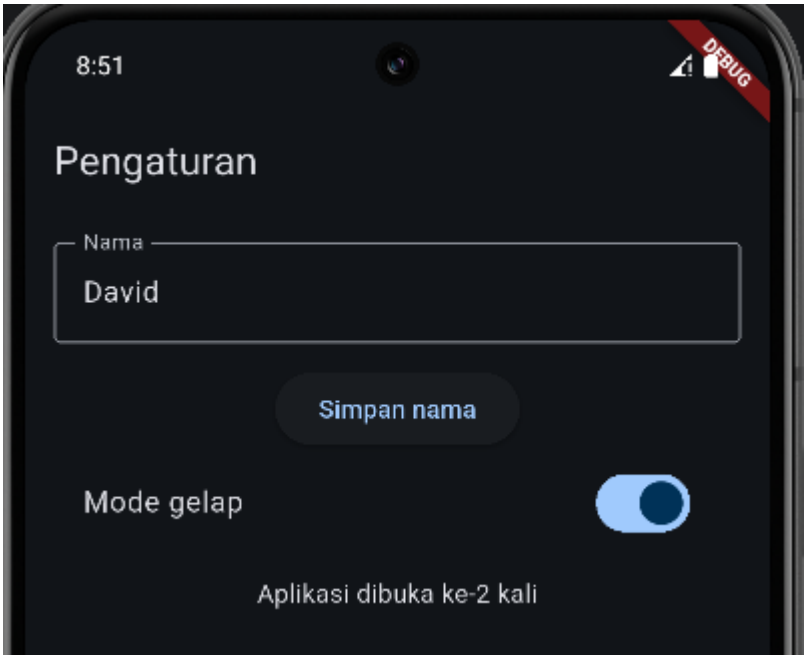
**Hasil :**


8:48
DEBUG
Pengaturan
Nama
Simpan nama
Mode gelap
Aplikasi dibuka ke-1 kali


8:49
DEBUG
Pengaturan
Nama
David
Simpan nama
Mode gelap
Aplikasi dibuka ke-1 kali


Nama disimpan
35%


8:49
DEBUG
Pengaturan
Nama
David
Simpan nama
Mode gelap
Aplikasi dibuka ke-1 kali


8:51
DEBUG
Pengaturan
Nama
David
Simpan nama
Mode gelap
Aplikasi dibuka ke-2 kali

## Bagian B :

```
PS C:\Users\advan\StudioProjects\untitled> flutter pub add sqflite path
```

```dart
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

class Catatan {
 final int? id;
 final String judul;
 final String isi;

 const Catatan({this.id, required this.judul, required this.isi});

 Map<String, Object?> toMap() => {'id': id, 'judul': judul, 'isi': isi};

 factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
   id: m['id'] as int?,
   judul: m['judul'] as String,
   isi: m['isi'] as String,
 );
}

class DbHelper {
 static Database? _db;

 static Future<Database> get database async {
   if (_db != null) return _db!;
   final path = p.join(await getDatabasesPath(), 'catatan.db');
   _db = await openDatabase(
     path,
     version: 1,
     onCreate: (db, version) {
       return db.execute(
         'CREATE TABLE catatan('
             'id INTEGER PRIMARY KEY AUTOINCREMENT, '
             'judul TEXT NOT NULL, '
             'isi TEXT NOT NULL)',
       );
     },
   );
   return _db!;
 }

 static Future<int> tambah(Catatan c) async {
   final db = await database;
   return db.insert('catatan', c.toMap());
 }

 static Future<List<Catatan>> semua() async {
   final db = await database;
   final rows = await db.query('catatan', orderBy: 'id DESC');
```

```
    return rows.map(Catatan.fromMap).toList();
  }

  static Future<int> ubah(Catatan c) async {
    final db = await database;
    return db.update('catatan', c.toMap(), where: 'id = ?', whereArgs:
[c.id]);
  }

  static Future<int> hapus(int id) async {
    final db = await database;
    return db.delete('catatan', where: 'id = ?', whereArgs: [id]);
  }
}
```

## Bagian C :

```dart
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() {
 runApp(const MyApp());
}

class MyApp extends StatelessWidget {
 const MyApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     debugShowCheckedModeBanner: false,
     title: 'Catatan Saya',
     theme: ThemeData(
       colorSchemeSeed: Colors.blue,
       useMaterial3: true,
     ),
     home: const CatatanPage(),
   );
 }
}

// ============================
// BAGIAN B - MODEL
// ============================

class Catatan {
 final int? id;
 final String judul;
 final String isi;

 const Catatan({
   this.id,
   required this.judul,
   required this.isi,
 });

 Map<String, Object?> toMap() {
   return {
     'id': id,
     'judul': judul,
     'isi': isi,
   };
 }

 factory Catatan.fromMap(Map<String, Object?> m) {
   return Catatan(
     id: m['id'] as int,
     judul: m['judul'] as String,
```

```
      isi: m['isi'] as String,
    );
  }
}

// ========================
// BAGIAN B - DATABASE
// ========================

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) {
      return _db!;
    }

    final path = p.join(
      await getDatabasesPath(),
      'catatan.db',
    );

    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'judul TEXT NOT NULL, '
              'isi TEXT NOT NULL)',
        );
      },
    );

    return _db!;
  }

  // READ
  static Future<List<Catatan>> semua() async {
    final db = await database;

    final rows = await db.query(
      'catatan',
      orderBy: 'id DESC',
    );

    return rows.map(Catatan.fromMap).toList();
  }

  // DELETE
  static Future<int> hapus(int id) async {
    final db = await database;
```

```
    return db.delete(
      'catatan',
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}

// ==========================
// BAGIAN C - LIST CATATAN
// ==========================

class CatatanPage extends StatefulWidget {
  const CatatanPage({super.key});

  @override
  State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;

  @override
  void initState() {
    super.initState();

    _future = DbHelper.semua();
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catatan Saya'),
      ),

      body: FutureBuilder<List<Catatan>>(
        future: _future,

        builder: (context, snapshot) {
          // Loading
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(
              child: CircularProgressIndicator(),
            );
          }
```

```
        // Error
        if (snapshot.hasError) {
          return Center(
            child: Text(
              'Galat: ${snapshot.error}',
            ),
          );
        }

        final data = snapshot.data ?? [];

        // Belum ada data
        if (data.isEmpty) {
          return const Center(
            child: Text(
              'Belum ada catatan',
              style: TextStyle(
                fontSize: 18,
              ),
            ),
          );
        }

        // Menampilkan data
        return ListView.builder(
          itemCount: data.length,

          itemBuilder: (context, i) {
            final c = data[i];

            return ListTile(
              title: Text(c.judul),

              subtitle: Text(
                c.isi,
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
              ),

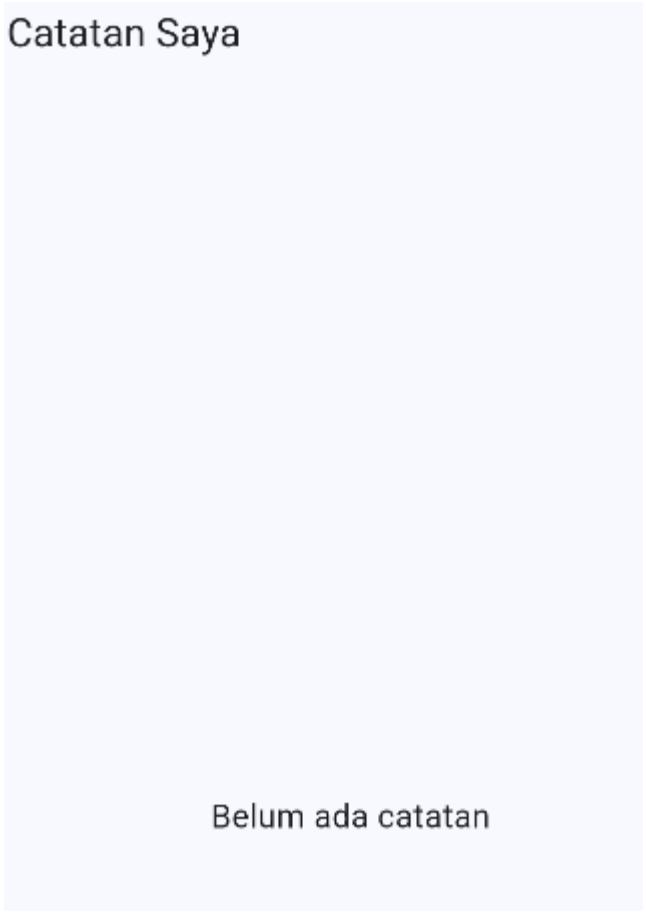
              // DELETE
              trailing: IconButton(
                icon: const Icon(Icons.delete),

                onPressed: () async {
                  await DbHelper.hapus(c.id!);

                  _muat();
                },
              ),
            );
          },
        );
```

```
        },
      ),
    );
  }
}
```

**Hasil :**



## Bagian D :

```dart
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

void main() { runApp(const MyApp());
}

class MyApp extends StatelessWidget {
 const MyApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(

     debugShowCheckedModeBanner: false,
     title: 'Catatan Saya',
     theme: ThemeData(
       colorSchemeSeed: Colors.blue,
       useMaterial3: true,
     ),
     home: const CatatanPage(),
   );
 }
}

class Catatan {
 final int? id;
 final String judul;
 final String isi;

 const Catatan({
   this.id,
   required this.judul,
   required this.isi,
 });

 Map<String, Object?> toMap() {
   return {
     'id': id,
     'judul': judul,
     'isi': isi,
   };
 }

 factory Catatan.fromMap(Map<String, Object?> m) {
   return Catatan(
     id: m['id'] as int,
     judul: m['judul'] as String,
     isi: m['isi'] as String,
   );
 }
}
```

```dart
class DbHelper {
 static Database? _db;

 static Future<Database> get database async {
   if (_db != null) {
     return _db!;
   }

   final path = p.join(
     await getDatabasesPath(),
     'catatan.db',
   );

   _db = await openDatabase(
     path,
     version: 1,
     onCreate: (db, version) {
       return db.execute(
         'CREATE TABLE catatan('
             'id INTEGER PRIMARY KEY AUTOINCREMENT, '
             'judul TEXT NOT NULL, '
             'isi TEXT NOT NULL)',
       );
     },
   );

   return _db!;
 }

 static Future<int> tambah(Catatan c) async {
   final db = await database;

   return db.insert(
     'catatan',
     c.toMap(),
   );
 }

 static Future<List<Catatan>> semua() async {
   final db = await database;

   final rows = await db.query(
     'catatan',
     orderBy: 'id DESC',
   );

   return rows.map(Catatan.fromMap).toList();
 }

 static Future<int> ubah(Catatan c) async {
   final db = await database;
```

```
    return db.update(
      'catatan',
      c.toMap(),
      where: 'id = ?',
      whereArgs: [c.id],
    );
  }

  static Future<int> hapus(int id) async {
    final db = await database;

    return db.delete(
      'catatan',
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}

class CatatanPage extends StatefulWidget {
  const CatatanPage({super.key});

  @override
  State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
  late Future<List<Catatan>> _future;

  @override
  void initState() {
    super.initState();

    _future = DbHelper.semua();
  }

  void _muat() {
    setState(() {
      _future = DbHelper.semua();
    });
  }

  Future<void> _buka([Catatan? catatan]) async {
    await Navigator.push(
      context,
      MaterialPageRoute(
        builder: (_) => FormCatatanPage(
          catatan: catatan,
        ),
      ),
    );

    if (!mounted) return;
```

```
  _muat();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Catatan Saya'),
    ),

    body: FutureBuilder<List<Catatan>>(
      future: _future,

      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        if (snapshot.hasError) {
          return Center(
            child: Text(
              'Galat: ${snapshot.error}',
            ),
          );
        }

        final data = snapshot.data ?? [];

        if (data.isEmpty) {
          return const Center(
            child: Text(
              'Belum ada catatan',
              style: TextStyle(
                fontSize: 18,
              ),
            ),
          );
        }

        return ListView.builder(
          itemCount: data.length,

          itemBuilder: (context, i) {
            final c = data[i];

            return ListTile(
              title: Text(c.judul),

              subtitle: Text(
                c.isi,
```

```
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
              ),

              onTap: () => _buka(c),

              trailing: IconButton(
                icon: const Icon(Icons.delete),

                onPressed: () async {
                  await DbHelper.hapus(c.id!);

                  _muat();
                },
              ),
            );
          },
        );
      },
    ),

    floatingActionButton: FloatingActionButton(
      onPressed: () => _buka(),

      child: const Icon(Icons.add),
    ),
  );
 }
}

class FormCatatanPage extends StatefulWidget {
 final Catatan? catatan;

 const FormCatatanPage({
   super.key,
   this.catatan,
 });

 @override
 State<FormCatatanPage> createState() => _FormCatatanPageState();
}

class _FormCatatanPageState extends State<FormCatatanPage> {
 final _formKey = GlobalKey<FormState>();

 late final TextEditingController _judul;
 late final TextEditingController _isi;

 @override
 void initState() {
   super.initState();

   _judul = TextEditingController(
```

```
    text: widget.catatan?.judul ?? '',
  );

  _isi = TextEditingController(
    text: widget.catatan?.isi ?? '',
  );
}

@override
void dispose() {
  _judul.dispose();
  _isi.dispose();

  super.dispose();
}

Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  final c = Catatan(
    id: widget.catatan?.id,
    judul: _judul.text.trim(),
    isi: _isi.text.trim(),
  );

  if (widget.catatan == null) {
    await DbHelper.tambah(c);
  } else {
    await DbHelper.ubah(c);
  }

  if (!mounted) return;

  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.catatan == null;

  return Scaffold(
    appBar: AppBar(
      title: Text(
        baru ? 'Catatan Baru' : 'Ubah Catatan',
      ),
    ),

    body: Form(
      key: _formKey,

      child: ListView(
```

```
        padding: const EdgeInsets.all(16),

        children: [
          TextFormField(
            controller: _judul,

            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),

            validator: (v) {
              if (v == null || v.trim().isEmpty) {
                return 'Judul wajib diisi';
              }

              return null;
            },
          ),

          const SizedBox(height: 12),

          TextFormField(
            controller: _isi,

            maxLines: 6,

            decoration: const InputDecoration(
              labelText: 'Isi catatan',
              border: OutlineInputBorder(),
            ),

            validator: (v) {
              if (v == null || v.trim().isEmpty) {
                return 'Isi wajib diisi';
              }

              return null;
            },
          ),

          const SizedBox(height: 16),

          ElevatedButton(
            onPressed: _simpan,

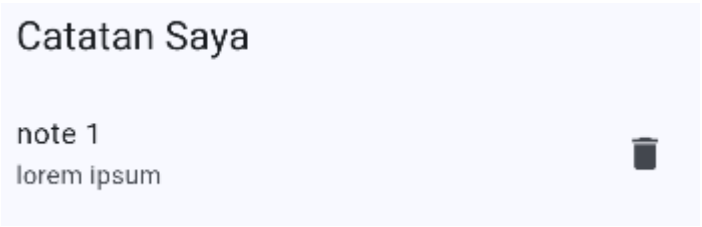
            child: const Text('Simpan'),
          ),
        ],
      ),
    ),
  );
}
```

```
}
```

**Hasil :**




Catatan Baru
Judul
Isi catatan
Simpan


Catatan Saya
note 1
lorem ipsum

## Latihan Mandiri :

```dart
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() {
 runApp(const MyApp());
}

class MyApp extends StatelessWidget {
 const MyApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     debugShowCheckedModeBanner: false,
     title: 'Catatan Saya',
     theme: ThemeData(
       colorSchemeSeed: Colors.blue,
       useMaterial3: true,
     ),
     home: const CatatanPage(),
   );
 }
}

class Catatan {
 final int? id;
 final String judul;
 final String isi;
 final String created;

 const Catatan({
   this.id,
   required this.judul,
   required this.isi,
   required this.created,
 });

 Map<String, Object?> toMap() {
   return {
     'id': id,
     'judul': judul,
     'isi': isi,
     'created': created,
   };
 }

 factory Catatan.fromMap(Map<String, Object?> m) {
   return Catatan(
     id: m['id'] as int,
     judul: m['judul'] as String,
```

```
      isi: m['isi'] as String,
      created: m['created'] as String,
    );
  }
}

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) {
      return _db!;
    }

    final path = p.join(
      await getDatabasesPath(),
      'catatan.db',
    );

    _db = await openDatabase(
      path,
      version: 2,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE catatan('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'judul TEXT NOT NULL, '
              'isi TEXT NOT NULL, '
              'created TEXT NOT NULL)',
        );
      },
      onUpgrade: (db, oldVersion, newVersion) async {
        if (oldVersion < 2) {
          await db.execute(
            'ALTER TABLE catatan '
                'ADD COLUMN created TEXT NOT NULL DEFAULT ""',
          );
        }
      },
    );

    return _db!;
  }

  static Future<int> tambah(Catatan c) async {
    final db = await database;

    return db.insert(
      'catatan',
      c.toMap(),
    );
  }
```

```dart
 static Future<List<Catatan>> semua({
   String keyword = '',
   bool terbaru = true,
 }) async {
   final db = await database;

   final rows = await db.query(
     'catatan',
     where: keyword.isEmpty
         ? null
         : 'judul LIKE ? OR isi LIKE ?',
     whereArgs: keyword.isEmpty
         ? null
         : ['%$keyword%', '%$keyword%'],
     orderBy: terbaru
         ? 'created DESC'
         : 'created ASC',
   );

   return rows.map(Catatan.fromMap).toList();
 }

 static Future<int> ubah(Catatan c) async {
   final db = await database;

   return db.update(
     'catatan',
     c.toMap(),
     where: 'id = ?',
     whereArgs: [c.id],
   );
 }

 static Future<int> hapus(int id) async {
   final db = await database;

   return db.delete(
     'catatan',
     where: 'id = ?',
     whereArgs: [id],
   );
 }
}

class CatatanPage extends StatefulWidget {
 const CatatanPage({super.key});

 @override
 State<CatatanPage> createState() => _CatatanPageState();
}

class _CatatanPageState extends State<CatatanPage> {
 late Future<List<Catatan>> _future;
```

```dart
final TextEditingController _searchController =
TextEditingController();

bool _terbaru = true;

@override
void initState() {
  super.initState();

  _loadSortPreference();
  _muat();
}

Future<void> _loadSortPreference() async {
  final prefs = await SharedPreferences.getInstance();

  final terbaru = prefs.getBool('sort_terbaru');

  if (terbaru != null) {
    setState(() {
      _terbaru = terbaru;
    });
  }

  _muat();
}

Future<void> _saveSortPreference(bool value) async {
  final prefs = await SharedPreferences.getInstance();

  await prefs.setBool(
    'sort_terbaru',
    value,
  );
}

void _muat() {
  _future = DbHelper.semua(
    keyword: _searchController.text.trim(),
    terbaru: _terbaru,
  );
}

void _refresh() {
  setState(() {
    _muat();
  });
}

void _search(String value) {
  setState(() {
    _future = DbHelper.semua(
```

```
      keyword: value.trim(),
      terbaru: _terbaru,
    );
  });
}

Future<void> _ubahSorting() async {
  final value = !_terbaru;

  await _saveSortPreference(value);

  setState(() {
    _terbaru = value;
    _muat();
  });
}

Future<void> _buka([Catatan? catatan]) async {
  await Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => FormCatatanPage(
        catatan: catatan,
      ),
    ),
  );

  if (!mounted) return;

  _refresh();
}

Future<void> _konfirmasiHapus(Catatan catatan) async {
  final result = await showDialog<bool>(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text('Hapus Catatan?'),
        content: Text(
          'Apakah kamu yakin ingin menghapus '
              '"${catatan.judul}"?',
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(context, false);
            },
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () {
              Navigator.pop(context, true);
            },
```

```
            child: const Text('Hapus'),
          ),
        ],
      );
    },
  );

  if (result != true) return;

  await DbHelper.hapus(catatan.id!);

  if (!mounted) return;

  _refresh();

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Catatan berhasil dihapus'),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Catatan Saya'),
      actions: [
        IconButton(
          tooltip: _terbaru
              ? 'Terlama ke Terbaru'
              : 'Terbaru ke Terlama',
          onPressed: _ubahSorting,
          icon: Icon(
            _terbaru
                ? Icons.arrow_downward
                : Icons.arrow_upward,
          ),
        ),
      ],
    ),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(16),
          child: TextField(
            controller: _searchController,
            onChanged: _search,
            decoration: InputDecoration(
              labelText: 'Cari catatan',
              hintText: 'Cari judul atau isi...',
              prefixIcon: const Icon(Icons.search),
              suffixIcon: _searchController.text.isNotEmpty
```

```
                ? IconButton(
              onPressed: () {
                _searchController.clear();
                _search('');
              },
              icon: const Icon(Icons.clear),
            )
                : null,
            border: const OutlineInputBorder(),
          ),
        ),
      ),
      Expanded(
        child: FutureBuilder<List<Catatan>>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState ==
                ConnectionState.waiting) {
              return const Center(
                child: CircularProgressIndicator(),
              );
            }

            if (snapshot.hasError) {
              return Center(
                child: Text(
                  'Galat: ${snapshot.error}',
                ),
              );
            }

            final data = snapshot.data ?? [];

            if (data.isEmpty) {
              return const Center(
                child: Text(
                  'Belum ada catatan',
                  style: TextStyle(
                    fontSize: 18,
                  ),
                ),
              );
            }

            return ListView.builder(
              itemCount: data.length,
              itemBuilder: (context, i) {
                final c = data[i];

                return ListTile(
                  title: Text(c.judul),
                  subtitle: Text(
                    '${c.isi}\nDibuat: ${c.created}',
```

```
                      maxLines: 3,
                      overflow: TextOverflow.ellipsis,
                    ),
                    onTap: () => _buka(c),
                    trailing: IconButton(
                      icon: const Icon(
                        Icons.delete,
                      ),
                      onPressed: () {
                        _konfirmasiHapus(c);
                      },
                    ),
                  );
                },
              );
            },
          ),
        ),
      ],
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () => _buka(),
      child: const Icon(
        Icons.add,
      ),
    ),
  );
 }

 @override
 void dispose() {
   _searchController.dispose();
   super.dispose();
 }
}

class FormCatatanPage extends StatefulWidget {
 final Catatan? catatan;

 const FormCatatanPage({
   super.key,
   this.catatan,
 });

 @override
 State<FormCatatanPage> createState() =>
     _FormCatatanPageState();
}

class _FormCatatanPageState
   extends State<FormCatatanPage> {
 final _formKey = GlobalKey<FormState>();
```

```dart
late final TextEditingController _judul;
late final TextEditingController _isi;

@override
void initState() {
  super.initState();

  _judul = TextEditingController(
    text: widget.catatan?.judul ?? '',
  );

  _isi = TextEditingController(
    text: widget.catatan?.isi ?? '',
  );
}

@override
void dispose() {
  _judul.dispose();
  _isi.dispose();

  super.dispose();
}

Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  final c = Catatan(
    id: widget.catatan?.id,
    judul: _judul.text.trim(),
    isi: _isi.text.trim(),
    created: widget.catatan?.created ??
        DateTime.now().toIso8601String(),
  );

  if (widget.catatan == null) {
    await DbHelper.tambah(c);
  } else {
    await DbHelper.ubah(c);
  }

  if (!mounted) return;

  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.catatan == null;

  return Scaffold(
```

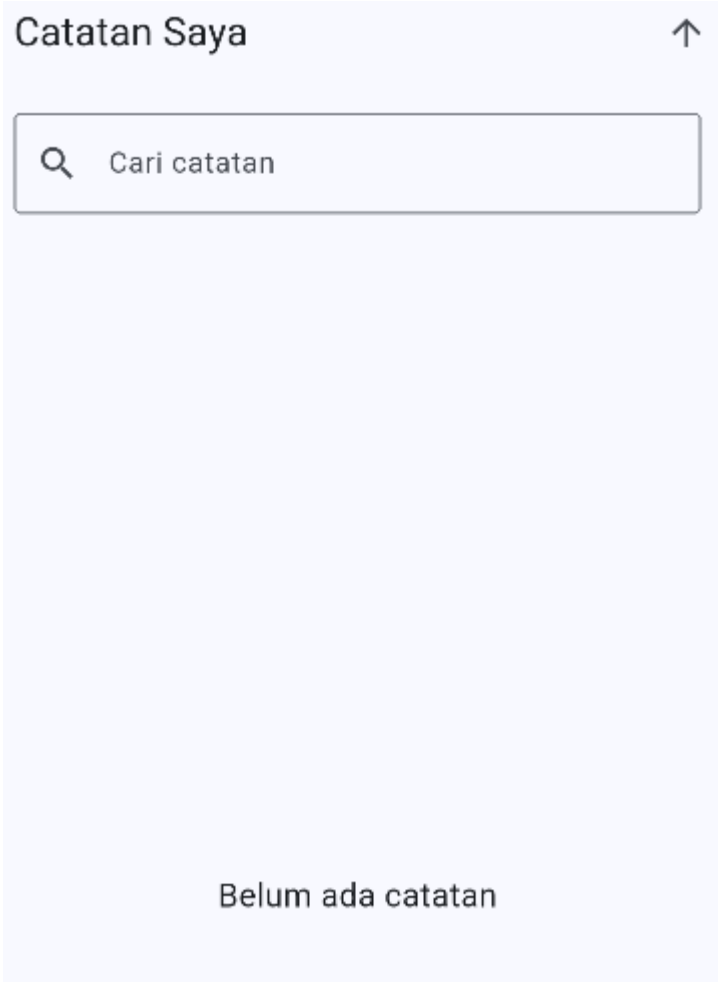
```
    appBar: AppBar(
      title: Text(
        baru
            ? 'Catatan Baru'
            : 'Ubah Catatan',
      ),
    ),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _judul,
            decoration: const InputDecoration(
              labelText: 'Judul',
              border: OutlineInputBorder(),
            ),
            validator: (v) {
              if (v == null ||
                  v.trim().isEmpty) {
                return 'Judul wajib diisi';
              }

              return null;
            },
          ),
          const SizedBox(height: 12),
          TextFormField(
            controller: _isi,
            maxLines: 6,
            decoration: const InputDecoration(
              labelText: 'Isi catatan',
              border: OutlineInputBorder(),
            ),
            validator: (v) {
              if (v == null ||
                  v.trim().isEmpty) {
                return 'Isi wajib diisi';
              }

              return null;
            },
          ),
          const SizedBox(height: 16),
          ElevatedButton(
            onPressed: _simpan,
            child: const Text('Simpan'),
          ),
        ],
      ),
    ),
  );
```

```
  }
}
```

**Hasil :**

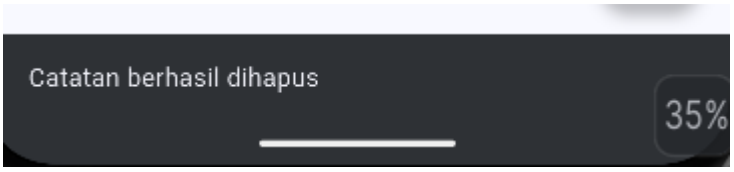


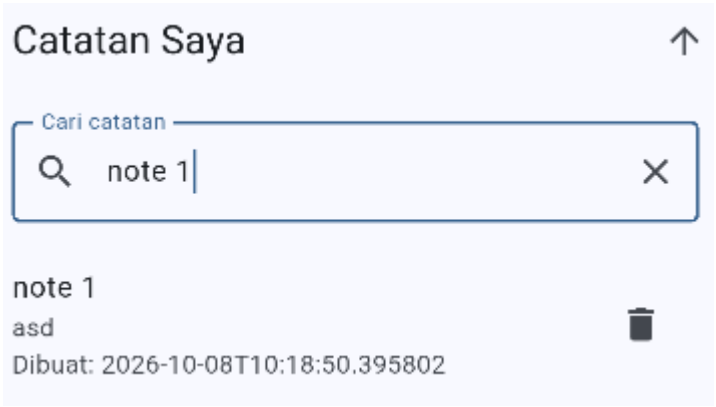

Catatan Baru
Judul
Isi catatan
Simpan


Catatan Saya
Cari catatan
note 2
sd
Dibuat: 2026-10-08T10:16:54.664539
note 1
asd
Dibuat: 2026-10-08T10:16:42.498918


Catatan Saya
Cari catatan
note 1
asd
Dibuat: 2026-10-08T10:16:42.498918
note 2
sd
Dibuat: 2026-10-08T10:16:54.664539


Hapus Catatan?
Apakah kamu yakin ingin menghapus "note 1"?
Batal
Hapus


Catatan berhasil dihapus
35%


Catatan Saya
Cari catatan
note 1
note 1
asd
Dibuat: 2026-10-08T10:18:50.395802

## Tugas :

```dart
import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() {
 runApp(const MyApp());
}

class MyApp extends StatelessWidget {
 const MyApp({super.key});

 @override
 Widget build(BuildContext context) {
   return MaterialApp(
     debugShowCheckedModeBanner: false,
     title: 'Expense Tracker',
     theme: ThemeData(
       colorSchemeSeed: Colors.blue,
       useMaterial3: true,
     ),
     home: const ExpensePage(),
   );
 }
}

class Expense {
 final int? id;
 final String name;
 final double amount;
 final String category;
 final String date;

 const Expense({
   this.id,
   required this.name,
   required this.amount,
   required this.category,
   required this.date,
 });

 Map<String, Object?> toMap() {
   return {
     'id': id,
     'name': name,
     'amount': amount,
     'category': category,
     'date': date,
   };
 }

 factory Expense.fromMap(Map<String, Object?> map) {
```

```dart
    return Expense(
      id: map['id'] as int,
      name: map['name'] as String,
      amount: (map['amount'] as num).toDouble(),
      category: map['category'] as String,
      date: map['date'] as String,
    );
  }
}

class DbHelper {
  static Database? _db;

  static Future<Database> get database async {
    if (_db != null) {
      return _db!;
    }

    final path = p.join(
      await getDatabasesPath(),
      'expense_tracker.db',
    );

    _db = await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) {
        return db.execute(
          'CREATE TABLE expenses('
              'id INTEGER PRIMARY KEY AUTOINCREMENT, '
              'name TEXT NOT NULL, '
              'amount REAL NOT NULL, '
              'category TEXT NOT NULL, '
              'date TEXT NOT NULL)',
        );
      },
    );

    return _db!;
  }

  static Future<int> tambah(Expense expense) async {
    final db = await database;

    return db.insert(
      'expenses',
      expense.toMap(),
    );
  }

  static Future<List<Expense>> semua() async {
    final db = await database;
```

```
    final rows = await db.query(
      'expenses',
      orderBy: 'date DESC, id DESC',
    );

    return rows.map(Expense.fromMap).toList();
  }

  static Future<int> ubah(Expense expense) async {
    final db = await database;

    return db.update(
      'expenses',
      expense.toMap(),
      where: 'id = ?',
      whereArgs: [expense.id],
    );
  }

  static Future<int> hapus(int id) async {
    final db = await database;

    return db.delete(
      'expenses',
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}

class ExpensePage extends StatefulWidget {
  const ExpensePage({super.key});

  @override
  State<ExpensePage> createState() => _ExpensePageState();
}

class _ExpensePageState extends State<ExpensePage> {
  late Future<List<Expense>> _future;

  bool _showTotal = true;

  @override
  void initState() {
    super.initState();

    _future = DbHelper.semua();

    _loadPreference();
  }

  Future<void> _loadPreference() async {
    final prefs = await SharedPreferences.getInstance();
```

```
  final showTotal = prefs.getBool('show_total');

  if (showTotal != null) {
    setState(() {
      _showTotal = showTotal;
    });
  }
}

Future<void> _savePreference(bool value) async {
  final prefs = await SharedPreferences.getInstance();

  await prefs.setBool(
    'show_total',
    value,
  );
}

void _muat() {
  setState(() {
    _future = DbHelper.semua();
  });
}

Future<void> _bukaForm([Expense? expense]) async {
  await Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => ExpenseFormPage(
        expense: expense,
      ),
    ),
  );

  if (!mounted) return;

  _muat();
}

Future<void> _hapus(Expense expense) async {
  final konfirmasi = await showDialog<bool>(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text('Hapus Pengeluaran'),
        content: Text(
          'Yakin ingin menghapus "${expense.name}"?',
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(context, false);
```

```
            },
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () {
              Navigator.pop(context, true);
            },
            child: const Text('Hapus'),
          ),
        ],
      );
    },
  );

  if (konfirmasi != true) return;

  await DbHelper.hapus(expense.id!);

  if (!mounted) return;

  _muat();

  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Pengeluaran berhasil dihapus'),
    ),
  );
}

String _rupiah(double value) {
  return 'Rp ${value.toStringAsFixed(0)}';
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Expense Tracker'),
      actions: [
        IconButton(
          onPressed: () async {
            final value = !_showTotal;

            await _savePreference(value);

            setState(() {
              _showTotal = value;
            });
          },
          icon: Icon(
            _showTotal
                ? Icons.visibility
                : Icons.visibility_off,
```

```
          ),
        ),
      ],
    ),
    body: FutureBuilder<List<Expense>>(
      future: _future,
      builder: (context, snapshot) {
        if (snapshot.connectionState ==
            ConnectionState.waiting) {
          return const Center(
            child: CircularProgressIndicator(),
          );
        }

        if (snapshot.hasError) {
          return Center(
            child: Text(
              'Galat: ${snapshot.error}',
            ),
          );
        }

        final data = snapshot.data ?? [];

        double total = 0;

        for (final expense in data) {
          total += expense.amount;
        }

        if (data.isEmpty) {
          return Column(
            children: [
              if (_showTotal)
                Card(
                  margin: const EdgeInsets.all(16),
                  child: Padding(
                    padding: const EdgeInsets.all(20),
                    child: Row(
                      mainAxisAlignment:
                      MainAxisAlignment.spaceBetween,
                      children: [
                        const Text(
                          'Total Pengeluaran',
                          style: TextStyle(
                            fontSize: 16,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        Text(
                          _rupiah(total),
                          style: const TextStyle(
                            fontSize: 20,
```

```
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                  ],
                ),
              ),
            ),
          const Expanded(
            child: Center(
              child: Text(
                'Belum ada pengeluaran',
                style: TextStyle(
                  fontSize: 18,
                ),
              ),
            ),
          ),
        ],
      );
    }

    return Column(
      children: [
        if (_showTotal)
          Card(
            margin: const EdgeInsets.fromLTRB(
              16,
              16,
              16,
              8,
            ),
            child: Padding(
              padding: const EdgeInsets.all(20),
              child: Row(
                mainAxisAlignment:
                MainAxisAlignment.spaceBetween,
                children: [
                  const Text(
                    'Total Pengeluaran',
                    style: TextStyle(
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  Text(
                    _rupiah(total),
                    style: const TextStyle(
                      fontSize: 20,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ],
              ),
```

```
        ),
      ),
      Expanded(
        child: ListView.builder(
          itemCount: data.length,
          itemBuilder: (context, index) {
            final expense = data[index];

            return Card(
              margin: const EdgeInsets.symmetric(
                horizontal: 16,
                vertical: 6,
              ),
              child: ListTile(
                title: Text(
                  expense.name,
                  style: const TextStyle(
                    fontWeight: FontWeight.bold,
                  ),
                ),
                subtitle: Text(
                  '${expense.category} • ${expense.date}',
                ),
                trailing: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
                      _rupiah(expense.amount),
                      style: const TextStyle(
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                    PopupMenuButton<String>(
                      onSelected: (value) {
                        if (value == 'edit') {
                          _bukaForm(expense);
                        }

                        if (value == 'delete') {
                          _hapus(expense);
                        }
                      },
                      itemBuilder: (context) {
                        return const [
                          PopupMenuItem(
                            value: 'edit',
                            child: Text('Edit'),
                          ),
                          PopupMenuItem(
                            value: 'delete',
                            child: Text('Hapus'),
                          ),
                        ];
```

```
                            },
                          ),
                        ],
                      ),
                    ),
                  );
                },
              ),
            ),
          ],
        );
      },
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () => _bukaForm(),
      child: const Icon(Icons.add),
    ),
  );
 }
}

class ExpenseFormPage extends StatefulWidget {
 final Expense? expense;

 const ExpenseFormPage({
   super.key,
   this.expense,
 });

 @override
 State<ExpenseFormPage> createState() =>
     _ExpenseFormPageState();
}

class _ExpenseFormPageState
   extends State<ExpenseFormPage> {
 final _formKey = GlobalKey<FormState>();

 late final TextEditingController _nameController;
 late final TextEditingController _amountController;
 late String _category;
 late DateTime _date;

 final List<String> _categories = [
   'Makanan',
   'Transportasi',
   'Belanja',
   'Hiburan',
   'Tagihan',
   'Lainnya',
 ];

 @override
```

```dart
void initState() {
  super.initState();

  _nameController = TextEditingController(
    text: widget.expense?.name ?? '',
  );

  _amountController = TextEditingController(
    text: widget.expense?.amount.toString() ?? '',
  );

  _category = widget.expense?.category ??
      _categories.first;

  _date = widget.expense != null
      ? DateTime.parse(widget.expense!.date)
      : DateTime.now();
}

@override
void dispose() {
  _nameController.dispose();
  _amountController.dispose();

  super.dispose();
}

String _formatDate(DateTime date) {
  final day = date.day.toString().padLeft(2, '0');
  final month = date.month.toString().padLeft(2, '0');
  final year = date.year.toString();

  return '$year-$month-$day';
}

Future<void> _pilihTanggal() async {
  final result = await showDatePicker(
    context: context,
    initialDate: _date,
    firstDate: DateTime(2020),
    lastDate: DateTime(2100),
  );

  if (result == null) return;

  setState(() {
    _date = result;
  });
}

Future<void> _simpan() async {
  if (!_formKey.currentState!.validate()) {
    return;
```

```
  }

  final amount = double.tryParse(
    _amountController.text.trim(),
  );

  if (amount == null || amount <= 0) {
    return;
  }

  final expense = Expense(
    id: widget.expense?.id,
    name: _nameController.text.trim(),
    amount: amount,
    category: _category,
    date: _formatDate(_date),
  );

  if (widget.expense == null) {
    await DbHelper.tambah(expense);
  } else {
    await DbHelper.ubah(expense);
  }

  if (!mounted) return;

  Navigator.pop(context);
}

@override
Widget build(BuildContext context) {
  final baru = widget.expense == null;

  return Scaffold(
    appBar: AppBar(
      title: Text(
        baru
            ? 'Tambah Pengeluaran'
            : 'Edit Pengeluaran',
      ),
    ),
    body: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          TextFormField(
            controller: _nameController,
            decoration: const InputDecoration(
              labelText: 'Nama Pengeluaran',
              border: OutlineInputBorder(),
            ),
            validator: (value) {
```

```
            if (value == null ||
                value.trim().isEmpty) {
              return 'Nama wajib diisi';
            }

            return null;
          },
        ),
        const SizedBox(height: 16),
        TextFormField(
          controller: _amountController,
          keyboardType:
          const TextInputType.numberWithOptions(
            decimal: true,
          ),
          decoration: const InputDecoration(
            labelText: 'Jumlah',
            prefixText: 'Rp ',
            border: OutlineInputBorder(),
          ),
          validator: (value) {
            if (value == null ||
                value.trim().isEmpty) {
              return 'Jumlah wajib diisi';
            }

            final amount = double.tryParse(
              value.trim(),
            );

            if (amount == null) {
              return 'Jumlah harus berupa angka';
            }

            if (amount <= 0) {
              return 'Jumlah harus lebih dari 0';
            }

            return null;
          },
        ),
        const SizedBox(height: 16),
        DropdownButtonFormField<String>(
          value: _category,
          decoration: const InputDecoration(
            labelText: 'Kategori',
            border: OutlineInputBorder(),
          ),
          items: _categories.map((category) {
            return DropdownMenuItem(
              value: category,
              child: Text(category),
            );
```

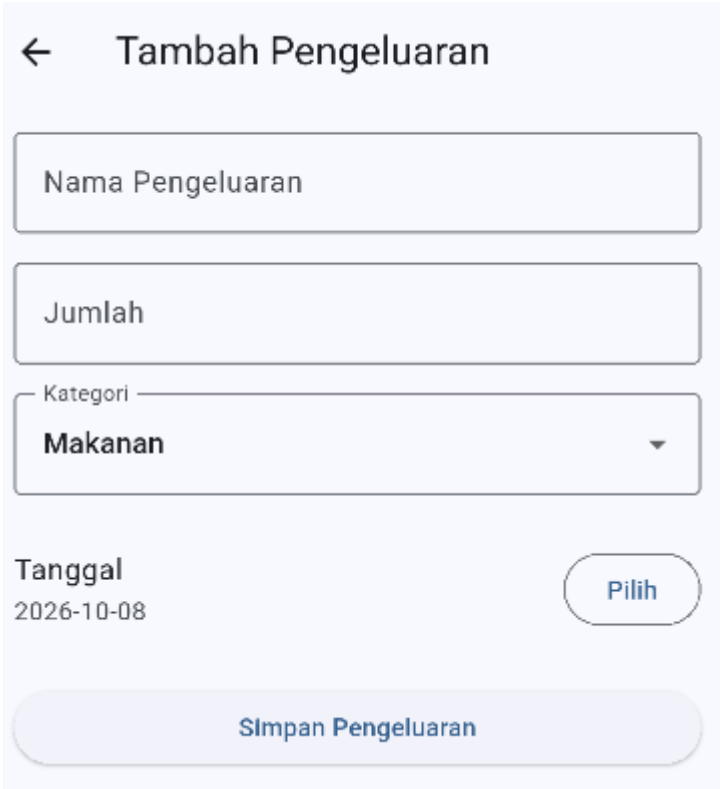
```
            }).toList(),
            onChanged: (value) {
              if (value == null) return;

              setState(() {
                _category = value;
              });
            },
          ),
          const SizedBox(height: 16),
          ListTile(
            contentPadding: EdgeInsets.zero,
            title: const Text('Tanggal'),
            subtitle: Text(
              _formatDate(_date),
            ),
            trailing: OutlinedButton(
              onPressed: _pilihTanggal,
              child: const Text('Pilih'),
            ),
          ),
          const SizedBox(height: 16),
          ElevatedButton(
            onPressed: _simpan,
            child: Text(
              baru
                  ? 'Simpan Pengeluaran'
                  : 'Simpan Perubahan',
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

**Hasil :**


Expense Tracker
Total Pengeluaran
Rp 0
Belum ada pengeluaran


Expense Tracker
Total Pengeluaran
Rp 0
Belum ada pengeluaran


Tambah Pengeluaran
Nama Pengeluaran
Jumlah
Kategori
Makanan
Tanggal
2026-10-08
Pilih
Simpan Pengeluaran

Makanan

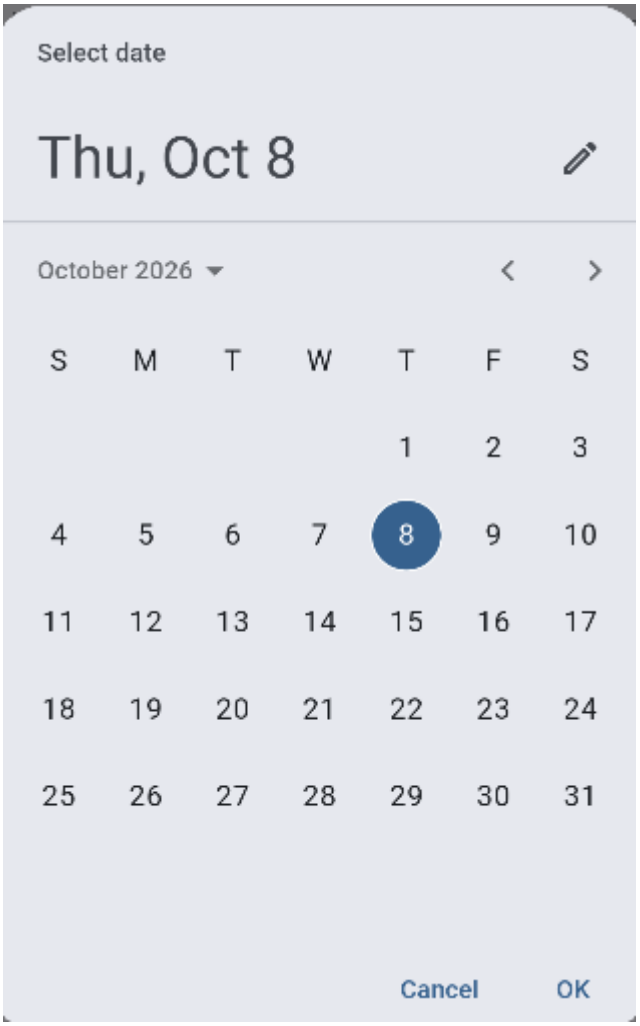
Transportasi

Belanja

Hiburan

Tagihan

Lainnya




Expense Tracker
Total Pengeluaran
Rp 30000
makan
Makanan • 2026-10-08
Rp 20000
minum
Hiburan • 2026-10-08
Rp 10000


Hapus Pengeluaran
Yakin ingin menghapus "makan"?
Batal
Hapus